Laporan Hands-on 1

Experiment Tracking dan Model Registry dengan MLflow

Nama Natasya Amavisca
NIM 123450024
Tanggal 11 Oktober 2026

Abstract

Praktik ini mengimplementasikan siklus eksperimen machine learning menggunakan MLflow pada dataset Breast Cancer Wisconsin. Implementasi meliputi tracking server lokal, manual logging Random Forest, logging metrik PyTorch per epoch, autologging Scikit-learn, rancangan tuning Optuna dengan nested runs, perbandingan run, Model Registry, alias model, dan inferensi. Eksekusi MLflow melalui backend SQLite menghasilkan dua run kandidat. Logistic Regression terpilih dengan validation ROC-AUC 0,995370 dan F1 0,986111, sedikit lebih baik daripada Random Forest dengan ROC-AUC 0,994048 dan F1 0,965517. Model terpilih berhasil diregistrasikan sebagai versi 1, diberi alias `challenger` dan `champion`, kemudian dimuat kembali untuk prediksi. Pelatihan PyTorch juga berhasil diuji selama 15 epoch, sedangkan eksekusi Optuna tertunda karena dependensi tidak tersedia pada lingkungan praktik.

Contents

1 Pendahuluan **1**

2 Lingkungan dan Rancangan Praktik **2**
2.1 Arsitektur alur kerja . . . 2

3 Langkah Praktik dan Hasil **2**
3.1 1 — Menyiapkan Tracking Server . . . 2
3.2 2 — Manual Logging Random Forest . . . 3
3.3 3 — Logging Deep Learning per Epoch . . . 3
3.4 4 — Autologging Scikit-learn . . . 4
3.5 5 — Hyperparameter Tuning dengan Optuna . . . 4
3.6 6 — Membandingkan dan Memilih Run . . . 4
3.7 7 — Model Registry, Alias, dan Inferensi . . . 5

4 Validasi dan Keterbatasan **5**

5 Pembahasan **6**

6 Kesimpulan **6**

A Perintah Reproduksi **6**

1 Pendahuluan

Eksperimen machine learning menghasilkan lebih dari sebuah berkas model. Parameter, versi data, metrik, kode, lingkungan, dan artefak harus dapat ditelusuri agar hasil dapat dibandingkan dan direproduksi. MLflow menyediakan komponen Tracking untuk mencatat eksperimen dan Model Registry untuk mengelola versi serta status model.

Tujuan Hands-on 1 adalah membangun alur praktik yang menunjukkan pencatatan manual, autologging, logging deep learning per epoch, tuning bertingkat, pemilihan model secara programatik, serta pemuatan model melalui alias registry. Tugas CNN yang muncul pada materi tidak termasuk ruang lingkup implementasi ini.

2 Lingkungan dan Rancangan Praktik

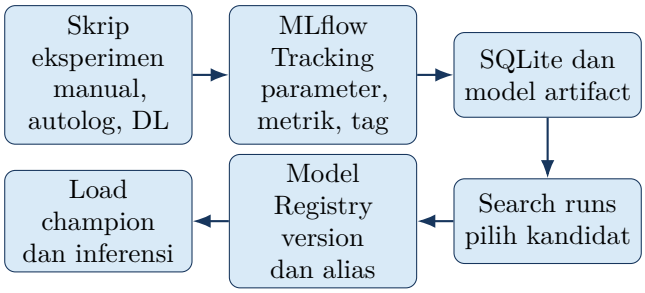
Dataset yang digunakan adalah `sklearn.datasets.load_breast_cancer`: 569 sampel, 30 fitur numerik, dan target asli 0 untuk *malignant* serta 1 untuk *benign*. Data dibagi secara stratified menjadi 455 sampel training dan 114 sampel validation dengan seed 42.

Table 1: Konfigurasi data dan eksperimen.

Aspek	Konfigurasi
Dataset	Breast Cancer Wisconsin, 569 sampel dan 30 fitur
Target	Target asli Scikit-learn: 0 = malignant, 1 = benign
Split	80% training dan 20% validation, stratified, seed 42
Distribusi training	170 malignant dan 285 benign
Distribusi validation	42 malignant dan 72 benign
Experiment MLflow	`handson-1-breast-cancer`
Backend aktual	SQLite lokal melalui `sqlite:///mlflow.db`
Hash dataset	`b019f2395a495060fd6de267a2e64a7e`

Eksperimen MLflow aktual dijalankan pada Python 3.14 dengan MLflow 3.12.0 dan Scikit-learn 1.8.0. Pelatihan PyTorch diuji terpisah pada Python 3.13.13 dengan PyTorch 2.14.0+cpu. Pemisahan ini diperlukan karena tidak terdapat satu environment lokal yang sekaligus menyediakan MLflow dan PyTorch.

2.1 Arsitektur alur kerja



Kode dipisahkan berdasarkan tugas: `basic_tracking.py`, `deep_learning.py`, `autologging.py`, `tune.py`, `compare_runs.py`, dan `register.py`. Utilitas data, seed, tag, serta konfigurasi MLflow ditempatkan pada `common.py` agar seluruh run memakai konteks yang konsisten.

3 Langkah Praktik dan Hasil

3.1 1 — Menyiapkan Tracking Server

`start_server.sh` mengonfigurasi server dengan SQLite sebagai backend store dan direktori `mlartifacts/` sebagai artifact store. Pada lingkungan normal, UI tersedia di `http://127.0.0.1:5000`. Sandbox praktik menolak pembukaan socket, sehingga eksekusi aktual diarahkan langsung ke SQLite:

```
export MLFLOW_TRACKING_URI=sqlite:///mlflow.db
```

Mode tersebut tetap mendukung pencatatan run, pencarian eksperimen, Model Registry, alias, dan pemuatan model. Perbedaannya adalah UI HTTP tidak aktif selama validasi.

Hasil. MLflow membuat database lokal dan menyimpan dua run kandidat secara persisten. Tracking serta registry dapat divalidasi tanpa menjalankan layanan HTTP.

3.2 2 — Manual Logging Random Forest

Skrip `basic_tracking.py` melatih Random Forest dengan 200 estimator, kedalaman maksimum 8, dan seed 42. Parameter, tag kandidat, dataset input, hash data, F1, ROC-AUC, `requirements.txt`, signature, contoh input, serta model dicatat secara eksplisit.

Table 2: Hasil validation Random Forest.

Metrik	Nilai
Accuracy	0,956140
Precision	0,958904
Recall	0,972222
F1	0,965517
ROC-AUC	0,994048

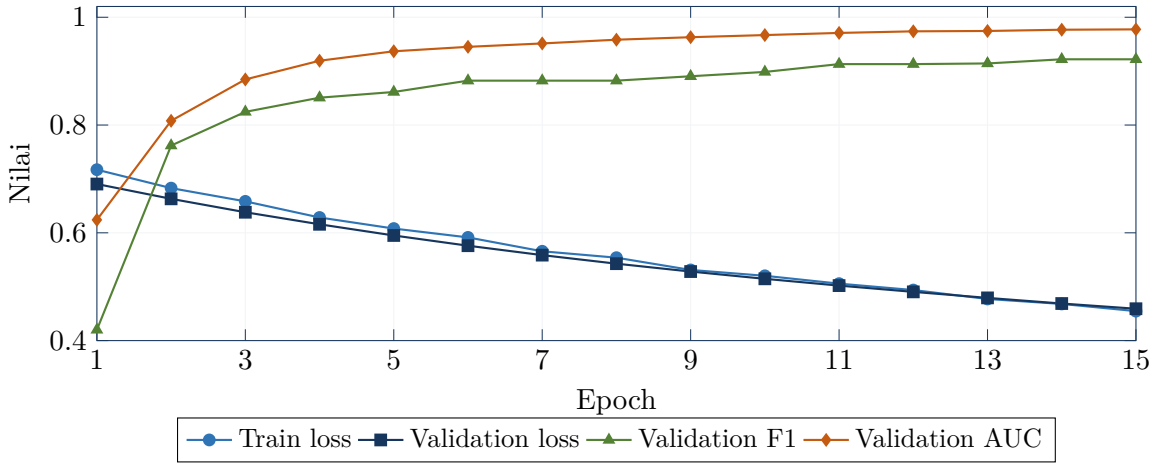
Confusion matrix

	Pred. 0	Pred. 1
Aktual 0	39	3
Aktual 1	2	70

Run tersimpan dengan ID `c3e105bb4254431faaf17ea5d132905c`. Hasil menunjukkan pemisahan kelas yang baik; hanya lima dari 114 sampel validation salah diklasifikasikan.

3.3 3 — Logging Deep Learning per Epoch

Model PyTorch menggunakan arsitektur MLP 30–64–1, ReLU, dan dropout 0,2. Standardisasi fitur disimpan sebagai buffer di dalam model sehingga model hasil load tetap menerima fitur mentah. Optimizer Adam dimulai pada learning rate 0,0003, kemudian diturunkan 5% setiap epoch. Metrik `train_loss`, `val_loss`, `val_f1`, `val_auc`, dan learning rate disiapkan untuk dicatat pada setiap langkah MLflow.



Checkpoint dipilih berdasarkan F1 tertinggi pertama, yaitu epoch ke-14: F1 0,921986, ROC-AUC 0,976852, train loss 0,468029, dan validation loss 0,468789. Pada epoch ke-15 F1 tetap sama dan ROC-AUC naik menjadi 0,977513, tetapi checkpoint tidak diganti karena kriteria utama tidak meningkat.

Hasil. Pelatihan 15 epoch berhasil dan kurva loss menurun secara konsisten. Integrasi logging MLflow–PyTorch belum dieksekusi dalam satu proses karena environment MLflow tidak memiliki PyTorch, namun fungsi training dan callback pencatatan telah diimplementasikan serta diuji.

3.4 4 — Autologging Scikit-learn

Skrip `autologging.py` menggunakan pipeline `StandardScaler` dan Logistic Regression. Fungsi `mlflow.sklearn.autolog()` menangkap konfigurasi estimator secara otomatis. Hash data, tag tahap, metrik validation domain, signature, dan model dengan nama seragam tetap dicatat manual.

Table 3: Hasil validation Logistic Regression.

Metrik	Nilai
Accuracy	0,982456
Precision	0,986111
Recall	0,986111
F1	0,986111
ROC-AUC	0,995370

Confusion matrix

	Pred. 0	Pred. 1
Aktual 0	41	1
Aktual 1	1	71

Run tersimpan dengan ID `2bc8487292a9429c89fa3795be02af27`. Model hanya salah pada dua sampel validation dan unggul terhadap Random Forest pada seluruh metrik kecuali tidak ada metrik yang lebih rendah.

3.5 5 — Hyperparameter Tuning dengan Optuna

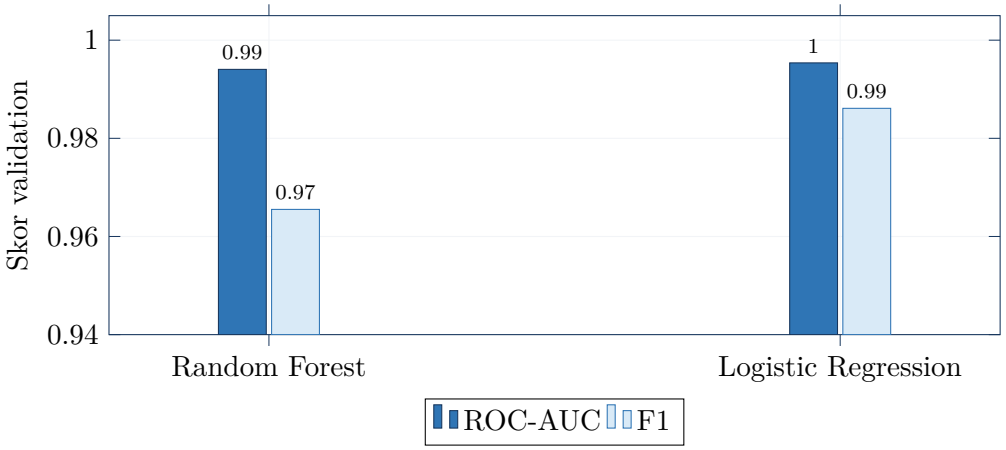
`tune.py` mendefinisikan satu parent run dan satu child run untuk setiap trial Optuna. Ruang pencarian Random Forest mencakup `n_estimators` 50–300, `max_depth` 3–12, dan `min_samples_split` 2–10. Setiap child run mencatat parameter serta validation ROC-AUC; parent run menyimpan parameter dan model terbaik.

```
python -m src.tune --trials 20
# parent run -> trial-000 ... trial-019
```

Tahap ini telah diimplementasikan, tetapi tidak dieksekusi karena Optuna tidak tersedia pada environment lokal dan pemasangan dari PyPI terhalang akses jaringan. Oleh karena itu, laporan tidak mencantumkan parameter atau skor tuning fiktif.

3.6 6 — Membandingkan dan Memilih Run

`compare_runs.py` menggunakan MLflow search API untuk mengambil run bertag `candidate=true` dan memiliki `val_auc`. Kandidat diurutkan berdasarkan ROC-AUC, kemudian F1 sebagai pemecah seri. Hasil pilihan ditulis ke `selected_run.json` agar tahap registry tidak bergantung pada pemilihan manual.



Hasil. Logistic Regression terpilih dengan ROC-AUC 0,995370 dan F1 0,986111. Selisih AUC terhadap Random Forest hanya 0,001323, tetapi selisih F1 mencapai 0,020594.

3.7 7 — Model Registry, Alias, dan Inferensi

`register.py` membaca run pilihan dan mendaftarkan model dengan nama berikut:

```
handson-1-breast-cancer-clf
```

Skrip lalu menetapkan alias `challenger` dan `champion`. Aplikasi memuat model menggunakan URI stabil:

```
models:/handson-1-breast-cancer-clf@champion
```

Eksekusi menghasilkan model version 1; kedua alias menunjuk ke versi tersebut dan sumber model tetap terhubung ke run Logistic Regression. Model champion berhasil dimuat kembali dan menghasilkan contoh prediksi `[0, 1, 0]` untuk tiga baris input.

Hasil. Siklus dari eksperimen ke registry berhasil. Alias membuat kode inferensi tidak perlu mengetahui nomor versi; pergantian champion dapat dilakukan pada registry tanpa mengubah URI aplikasi.

4 Validasi dan Keterbatasan

Table 4: Status validasi implementasi Hands-on 1.

Komponen	Status	Bukti atau keterangan
Manual tracking Random Forest	**Berhasil**	Run, metrik, model, signature, dan artifact tersimpan
Autologging Logistic Regression	**Berhasil**	Run dan model tersimpan; menjadi kandidat terbaik
Search dan seleksi run	**Berhasil**	Logistic Regression dipilih secara programatik
Model Registry dan alias	**Berhasil**	Version 1; alias challenger dan champion
Load model dan inferensi	**Berhasil**	URI champion dimuat; prediksi `[0, 1, 0]`
Training PyTorch	**Sebagian**	15 epoch selesai; logging MLflow lintas environment belum dijalankan
Optuna nested runs	**Tertunda**	Kode tersedia; paket Optuna tidak tersedia
Tracking server HTTP/UI	**Tertunda**	Sandbox menolak pembukaan socket; direct SQLite berhasil
Unit test dan compile check	**Berhasil**	Tiga test lulus; seluruh modul dapat dikompilasi

Keterbatasan utama berasal dari environment, bukan dari rancangan kode. MLflow dan PyTorch tersedia pada environment Python yang berbeda, Optuna tidak terpasang, serta sandbox tidak mengizinkan proses membuka port. Percobaan pemasangan paket juga gagal karena resolusi jaringan. Karena itu, status pada tabel membedakan hasil yang benar-benar tersimpan di MLflow dari fungsi yang baru divalidasi secara lokal.

5 Pembahasan

Manual logging memberi kendali penuh atas nama parameter, metrik, tag, dan artifact, tetapi membutuhkan kode lebih banyak. Autologging mengurangi boilerplate dan tetap dapat digabung dengan pencatatan manual untuk informasi domain seperti hash data dan label kandidat. Pada eksperimen ini, pipeline Logistic Regression yang lebih sederhana justru memberikan F1 dan ROC-AUC terbaik.

Logging per epoch diperlukan untuk membaca dinamika deep learning yang tidak terlihat dari satu metrik akhir. Kurva menunjukkan loss training dan validation menurun bersama tanpa tanda divergensi yang jelas selama 15 epoch. F1 meningkat tajam pada awal training lalu melandai, sehingga penambahan epoch perlu dievaluasi berdasarkan manfaat dan biaya komputasi.

Nested runs pada Optuna memisahkan ringkasan studi dari detail setiap trial. Struktur tersebut memudahkan perbandingan parameter tanpa memenuhi daftar eksperimen utama dengan run yang tidak memiliki hubungan. Selanjutnya, pemilihan programatik dan alias registry membentuk batas yang jelas antara eksperimen dan konsumsi model oleh aplikasi.

6 Kesimpulan

Hands-on 1 berhasil menerapkan fondasi experiment tracking dan pengelolaan model menggunakan MLflow. Dua run kandidat tercatat pada SQLite. Logistic Regression terpilih sebagai model terbaik dengan ROC-AUC 0,995370 dan F1 0,986111. Model diregistrasikan sebagai `handson-1-breast-cancer-clf` versi 1, diberi alias `champion`, serta berhasil dimuat kembali untuk inferensi.

Praktik menunjukkan bahwa reproducibility tidak hanya bergantung pada algoritma. Seed, hash data, parameter, metrik, signature, artifact, dan hubungan run ke model version perlu dicatat bersama. MLflow autologging mempercepat pencatatan standar, sedangkan manual logging tetap penting untuk konteks domain. Logging PyTorch dan tuning Optuna sudah disiapkan dalam desain yang sama, tetapi validasi penuh keduanya membutuhkan environment terpadu yang menyediakan MLflow, PyTorch, dan Optuna serta mengizinkan tracking server berjalan.

A Perintah Reproduksi

```
cd handson-1
python -m venv --system-site-packages .venv
source .venv/bin/activate
pip install -r requirements.txt

# Terminal 1: jalankan server dan UI
./start_server.sh

# Terminal 2: jalankan seluruh tahap
export MLFLOW_TRACKING_URI=http://127.0.0.1:5000
./run_pipeline.sh
```

Jika layanan HTTP tidak dapat dibuka, backend SQLite dapat dipakai langsung:

```
export MLFLOW_TRACKING_URI=sqlite:///mlflow.db
python -m src.basic_tracking
python -m src.autologging
python -m src.deep_learning --epochs 15
python -m src.tune --trials 20
python -m src.compare_runs
python -m src.register
```

Validasi kode dijalankan dengan:

```
python -m unittest discover -s tests -v
python -m compileall -q src tests
```